SEMIABELIAN GROUPS NEED NOT BE MONOMIAL

Joseph Phillip Brennan* and Milana Golich
with Muse Spark via meta.ai

Reviewed by
Andres Barei and John Portin

Abstract. We disprove a conjecture of M. Kida by exhibiting a finite semiabelian group of order 384 that is not monomial. The example is given by GAP's SmallGroups library as SmallGroup$(384, 20127)$ which is isomorphic to the group $((C_2 \times ((C_2 \times Q_8) \rtimes C_2)) \rtimes C_2) \rtimes C_3$. We remark on the use of AI in this research, see Statement of AI Use.

1. Introduction

Human

Semiabelian groups were first established within inverse Galois theory by Saltman [Sal82], Thompson [Tho84], and Matzat [Mat87] as a subclass of finite solvable groups that play a role in the solvability of split embedding problems with abelian kernel. This was further explored by Stoll [Sto95] and Dentzer [Den95] who showed that semiabelian groups occur as the Galois groups over Hilbertian fields. In [Leg22], Legrand extended this result to division rings by showing that for every division ring H and every automorphism σ of finite order, all semiabelian groups occur as the Galois group over the skew field of fractions $H(T, \sigma)$. It was over the course of these developments that the definition of semiabelian groups evolved into its present form:

Definition 1.1. A finite group G is called *semiabelian* if it has a sequence

$$G_0 = 1 \leq G_1 \leq \cdots \leq G_n = G$$

such that G_{i+1} is a quotient group of $G_i \ltimes A_i$ with a finite abelian group A_i for all $i = 0, ..., n-1$.

Elementary properties of semiabelian groups were derived naturally through the explicit derivation of semiabelian groups as Galois groups, but still much remained unknown about their group structure. Kida [Kid25] conducted a more thorough investigation into their group properties, but had found the recursive nature of their definition to be challenging in characterizing such groups. In fact, any inductive argument to characterize would fail in this regard since subgroups of semiabelian groups do not inherit the semiabelian property. Thus as a means to assist in classification, Kida uses the machinery of P. Hall [Hal40] to prove that semiabelian groups are invariant under isoclinism class. Here Kida suggests a strong connection with monomial groups, also invariant under isoclinism class, and derived further results in [Kid25] to support this. This led to the following conjecture:

Conjecture 1.2 (Kida)**.** *If a group G is semiabelian, then G is monomial.*

A small computational check in support of the conjecture was performed via the computer alge-

2020 *Mathematics Subject Classification.* 20D10, 20C15.
*Supporting author

Human bra system Magma up to groups of order 240 [Kid25]. Specifically, the Magma code unsuccessfully found counterexamples after implementing the following simple search algorithm: identify all non-monomial groups up to order 240 and check if they are also semiabelian. The main goal of this article is to present our continued search strategy for counterexamples of groups that are of order greater than 240. As a part of our findings, we show the existence of a finite group of order 384 that serves as a counterexample to Conjecture 1.2. This gives the following main theorem:

Theorem 1.3. *Semiabelian groups need not be monomial.*

The counterexample was found via an extended search over the computer algebra system `GAP`'s [GAP26] SmallGroups library [BEO26] and can be identified as $G = \texttt{SmallGroup}(384, 20127)$ which is isomorphic to the group $((C_2 \times ((C_2 \times Q_8) \rtimes C_2)) \rtimes C_2) \rtimes C_3$. Throughout the remaining sections of this article, we present a literature review of semiabelian and monomial groups, our search strategy via `GAP`, and our computational findings.

Statement of AI Use

Human This paper was developed through collaboration between researchers and Muse Spark (AI model) via the meta.ai chat interface. The model helped explore ideas, develop candidate arguments, and draft material. Researchers guided the work, checked and corrected the mathematics, and take ownership for the final manuscript. Following [Sch25], margin markers identify human-drafted (**Human**) and AI-assisted material (**AI**) separately.

Concurrent Work

Human While in the process of completing this work, the authors became aware of an additional counterexample provided on September 16, 2026 by the AI agent Nilradical [Nil26]. In this article, we offer both a minimal counterexample and also theoretical results that were done independently from that of Nilradical.

2. Preliminaries

Human **2.1. Semiabelian Groups.** For the purpose of identifying semiabelian groups within the family of finite solvable groups, we present a few useful properties.

First, it is easily seen that abelian groups are semiabelian. For a nilpotent group to be semiabelian, the group must have nilpotency class of at least 2 [Tho84]. If a solvable group G is semiabelian, then it admits an abelian normal subgroup which is not contained in the Frattini subgroup $\Phi(G)$. These three conditions will be useful for our search strategy outlined in section 3.1.

To provide background on the derivation of Conjecture 1.2, we also explore a convenient link between semiabelian groups, modular groups, and quaternion-free groups, and recall their definitions here:

Definition 2.1 (Kida, Definition 3.1)**.** A finite p-group G is *modular* if the subgroup lattice of G is a modular lattice, namely, $\langle A, B\rangle \cap C = \langle A, B \cap C\rangle$ for subgroups A, B, C of G such that $A \leq C$.

Definition 2.2 (Kida, Definition 3.7)**.** A 2-group G is called *quaternion-free* if all homomorphic

Human

images of every subgroup of G do not contain a subgroup isomorphic to Q_8.

As it turns out, modular p-groups and quaternion-free 2-groups are semiabelian [Kid25]. It is from these subclasses that Kida forms the main connection between semiabelian groups and monomial groups, and we establish this in the next section.

2.2. Monomial Groups. We first recall the definition of a monomial group here:

Definition 2.3. A group is *monomial* if every irreducible representation of G can be obtained via induction from a one dimensional representation of some subgroup of G.

As a means to identify if a finite solvable group is monomial, Dornhoff [Dor67] proposed the following criterion in connection with modular p-groups and quaternion-free 2-groups:

Theorem 2.4 (Dornhoff, Theorem 2.6)**.** *If a solvable group G is such that*

1. *the Sylow p-subgroups are modular for $p \geq 3$, and*
2. *the Sylow 2-subgroup is quaternion-free,*

then G is monomial.

Kida suggests in Remark 5.2 of [Kid25] that an analogous statement to Theorem 2.4 may be said about solvable groups that are semiabelian, raising the following question: under what conditions should we expect the semiabelian and monomial properties to agree? We investigate this further in Section 2.3.

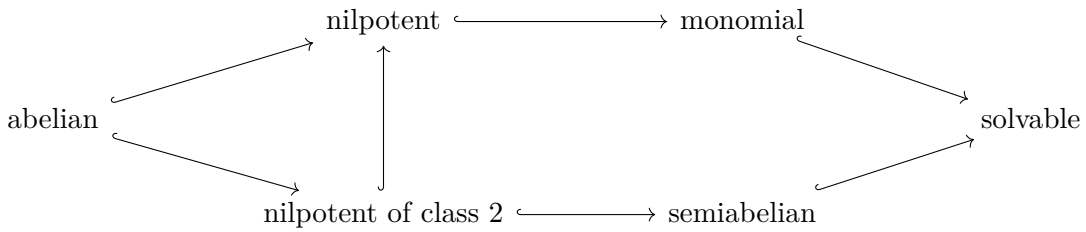
2.3. Isoclinism Classes and Conjecture 1.2. In an attempt to formally bridge the two properties, Kida turned to isoclinism classes, an equivalence relation first introduced by P. Hall to classify subclasses of solvable groups [Hal40].

Given a pair of groups G_1 and G_2, we say that they are *isoclinic* if there exists an isomorphism $\varphi : G_1/Z(G_1) \to G_2/Z(G_2)$ that induces an isomorphism $\psi : [G_1, G_1] \to [G_2, G_2]$ such that the following diagram commutes:

$$\begin{array}{ccc} G_1/Z(G_1) \times G_1/Z(G_1) & \xrightarrow{\varphi\times\varphi} & G_2/Z(G_2) \times G_2/Z(G_2) \\ \downarrow{\scriptstyle [\cdot,\cdot]_{G_1}} & & \downarrow{\scriptstyle [\cdot,\cdot]_{G_2}} \\ [G_1, G_1] & \xrightarrow{\psi} & [G_2, G_2] \end{array}$$

Furthermore, we say groups G_1 and G_2 lie in the same *isoclinism class* if G_1 is isoclinic to G_2. Isoclinism is therefore a weaker version of isomorphism. Informally, we think of two groups as isoclinic if their commutator structure is the same. For instance, all abelian groups are isoclinic to one another since their commutator is trivial, and D_8 and Q_8 are isoclinic but not isomorphic.

Many subclasses of finite solvable groups, including monomial groups, were first shown to be invariant under isoclinism in [BvdW78], and later extended to semiabelian groups in [Kid25]. As a result, we have the following hierarchy:



It was thus through this shared invariance that semiabelian groups were shown to admit a strong connection with monomial groups [Kid25]. The main result of which was the primary contributor to the statement of Conjecture 1.2:

Theorem 2.5 (Kida, Theorem 1.4)**.** *Let $\mathcal{A}$ be an isoclinism invariant class of finite groups with the property that if $G \in \mathcal{A}$, then every subgroup of G is contained in $\mathcal{A}$ and every homomorphic image of G is in $\mathcal{A}$. If $\mathcal{A}$ consists of semiabelian groups, then every $G \in \mathcal{A}$ is monomial.*

We note, however, that there is not an analogous result to Theorem 2.5 that would apply to the converse of Conjecture 1.2. In fact, it is well known that the converse does not hold true within the class of supersolvable groups. An explicit example of which was referenced by Kida as the group $G = \texttt{SmallGroup}(256, 330)$ ([Kid25], Example 4.8).

3. Search Strategy

In this section, we propose our two-fold search strategy: first verifying whether a group is semiabelian, then checking for non-monomiality.

3.1. Semiabelian Group Check. The first goal of our search strategy is to manually check if a group is semiabelian. Given the recursive nature of Definition 1.1, we aim to minimize the computational complexity of the search. Hence we perform our check over the following minimal characterization of semiabelian groups given by Dentzer [Den95]:

Lemma 3.1 (Dentzer, Theorem 2.3)**.** *A non-trivial finite group G is semiabelian if and only if there is an abelian normal subgroup A and a proper semiabelian subgroup M such that $G = AM$.*

Computationally, finding a candidate pair (A, M) that satisfies Lemma 3.1 would require enumerating all subgroups of G and computing their product with every normal abelian subgroup. However, we will show that it suffices to check only one class representative of each conjugacy class of maximal subgroups.

Lemma 3.2. *Let G be a finite semiabelian group that admits an abelian normal subgroup $A \lhd G$ and a proper semiabelian subgroup $U < G$ such that $G = AU$. Then for every maximal subgroup M with $U \leq M < G$, $G = AM$ and M is semiabelian.*

Proof. If U is maximal, we are done. So, assume that $U < M$ and let $L = M \cap A$. We will show that L is a normal abelian subgroup of M and that $M = (M \cap A)U$. Hence M is the product of a normal abelian subgroup and a semiabelian subgroup, which implies that M is semiabelian.

Let $m = au$ for some $m \in M$, $a \in A$, and $u \in U$. Then $a = mu^{-1} \in M$, so that $a \in M \cap A$. Given that $G = AU$ and $U \leq M$, the Dedekind modular law gives

$$M \cap G = M \cap AU = (M \cap A)U = LU.$$

AI

If follows that L is abelian, since $M \cap A \leq A$. To see that $L \lhd M$, let $m \in M$, $a \in L$ and consider mam^{-1}. Since $a \in L = M \cap A$, then $mam^{-1} \in M$. Further, since $A \lhd G$, $mam^{-1} \in A$. Thus $mam^{-1} \in L$ so that $mLm^{-1} \subseteq L$. Similarly, $mLm^{-1} \supseteq L$. Hence $L \lhd M$. □

For a group G and subset $K \subseteq G$, denote the conjugate of K by $g \in G$ as $K^g := \{g^{-1}kg : k \in K\}$. Semiabelian groups are also closed under conjugation.

Lemma 3.3. *Let G be a finite group and $M \leq G$ a maximal subgroup. Then M is semiabelian if and only if M^g is semiabelian for any $g \in G$. Further, if $G = AM$, then $G = AM^g$.*

Proof. Let $A \lhd G$ be a normal abelian group witnessing G being semiabelian, and let $g \in G$. Consider,

$$G = G^g = (AM)^g = A^g M^g = AM^g.$$

Further, since conjugates are isomorphic, M is semiabelian if and only if M^g is as well. □

Human

Muse Spark then assisted in generating the `GAP` code to implement Dentzer's lemma as a means to check if a group is semiabelian. The full code block used is given in Appendix A.1. We provide here a basic scheme that reflects the algorithm behind the code:

AI

Algorithm 1: Recursive Semiabelian Test

```
function IsSemiabelianGroup(G)
    Memoized by IdGroup
// 1. Check base cases
    n ← |G|
    if n = 1 or IsAbelian(G) then
        return true
    end if
    if ¬IsSolvable(G) then
        return false
    end if
    if IsNilpotent(G) and NilpotencyClass(G) ≤ 2 then
        return true
    end if

// 2. Check recursive definition: G = AM with M semiabelian
    Φ ← FrattiniSubgroup(G)
    A ← {N ⊴ G | 1 < |N| < n ∧ IsAbelian(N) ∧ N ⋦ Φ}
    if A = ∅ then return false
    end if
    M ← MaximalSubgroupClassReps(G)
    for A ∈ A do
        for M ∈ M do
            if |⟨A, M⟩| = n and IsSemiabelianGroup(M) then
                return true
            end if
        end for
```

AI

```
    end for
    return false
end function
```

Human

3.2. Monomial Group Check. To continue our search, we check if a group verified to be semiabelian is also non-monomial. Conveniently, we use GAP's built-in function `IsMonomial` to check if a group is non-monomial. This allows for a straightforward verification of monomiality in comparison to the semiabelian check outlined in Section 3.1.

4. GAP's SmallGroups example

Human

To complete our search, we prompted Muse Spark to generate a comprehensive semiabelian, non-monomial search algorithm in GAP. We ran the algorithm for groups up to order 900. For reference, we include the full code block used in Appendix A.2.

4.1. Counterexample to Conjecture 1.2. As a direct consequence of running the complete search algorithm in GAP, we received the following counterexample to Conjecture 1.2:

$$G = \texttt{SmallGroup}(384, 20127) = ((C_2 \times ((C_2 \times Q_8) \rtimes C_2)) \rtimes C_2) \rtimes C_3.$$

That is, the solvable group G of order 384 is semiabelian, but non-monomial. The decomposition of G is given by GAP's SmallGroups library [BEO26].

For reference, we cite the GAP output for our complete search algorithm in Appendix A.4.

4.2. Final Verification. In effort to fully verify that G satisfies Dentzer's lemma 3.1, we implemented a final check in GAP. The code is included in Appendix A.2 and the output in Appendix A.4. The recursive decomposition into the product of abelian and semiabelian groups is given as follows:

$$\begin{aligned} G = A_0M_0 &= (C_2 \times C_2 \times C_2)(((C_2 \times C_2 \times C_2) \rtimes (C_2 \times C_2)) \rtimes C_3) \\ M_0 = A_1M_1 &= (C_2 \times C_2 \times C_2)(C_2 \times A_4) \\ M_1 = A_2M_2 &= (C_2 \times C_2 \times C_2)(A_4) \\ M_2 = A_3M_3 &= (C_2 \times C_2)(C_3) \\ M_3 &= C_3. \end{aligned}$$

The groups A_i for $i = 0, 1, 2, 3$ are abelian since they are either cyclic groups or the direct product thereof. The initial group M_3 is semiabelian since it is abelian. Following the chain, the M_i for $i = 2, 1, 0$ are semiabelian using Definition 3.1. This finalizes our search and proves Theorem 1.3.

Remark 4.1. We note that one can build countably more semiabelian, non-monomial groups from this base example via direct products. In particular, we cite the following lemmas from Kida [Kid25] and W. van der Waall [vdW88]:

Lemma 4.2 (Kida, Lemma 2.5)**.** *If G and H are semiabelian groups, then the direct product $G \times H$ is semiabelian.*

Human

Lemma 4.3 (W. van der Waall, Theorem 2)**.** *The direct product $G \times H$ is monomial if and only if G and H are monomial.*

Hence we may simply take the direct product between $G = \texttt{SmallGroup}(384, 20127)$ and any semiabelian group H to build countably more examples that are semiabelian and non-monomial.

Remark 4.4. We also remark that this counterexample G serves as a stem group in its isoclinism class. That is, G satisfies $Z(G) \leq [G, G]$ and serves as a representative of its class.

5. Conclusion

Human

In our search over groups up to order 900, we did find a pair of groups of order 768 that are semiabelian and non-monomial, but they lie in the same isoclinism class as G. Beyond our search, the AI agent Nilradical [Nil26] had found another group of order 2592 that lies in this isoclinism class. Still, it remains unknown whether there exists examples unique to G. We hope that this motivates further study into the structure of such groups.

As a closing remark, one insight possibly worth studying further would be the conditions of Dornhoff's theorem given in Theorem 2.4. The main question of interest being the following: can Dornhoff's criteria be refined into necessary and sufficient conditions for a group to be both semiabelian and monomial? For the particular counterexample that we found, we have $G = P \rtimes C_3$ where P is a normal 2-subgroup of order 128 isomorphic to $(C_2 \times ((C_2 \times Q_8) \rtimes C_2)) \rtimes C_2$. Hence P is the unique Sylow 2-subgroup and contains a Q_8 subgroup. Consequently, G fails to satisfy condition (2) of Dornhoff's theorem.

Therefore, we conclude with the following question:

Question 5.1. Must all finite semiabelian, non-monomial groups admit a Sylow 2-subgroup that contains a group isomorphic to Q_8?

Acknowledgments

The authors would like to thank Ricky Simanjuntak for discussions at various stages of this project and his feedback on the final draft. The authors would also like to thank Andres Barei and John Portin for so kindly reviewing this work and offering very thoughtful suggestions and edits which have greatly improved the content of this draft.

Appendix A. GAP code

We performed the following computations in GAP 4.16.1:

AI

A.1. Semiabelian group check.

```
##############################################################################
# semiabelian.g
##############################################################################

if not IsBound(SEMIABELIAN_CACHE) then
  SEMIABELIAN_CACHE := rec();
fi;
```

```
ClearSemiabelianCache := function()
  SEMIABELIAN_CACHE := rec();
end;

SemiabelianCacheKey := function(G)
  local r, id;
  if Size(G) > 2000 or Size(G) = 1024 then
    return fail;
  fi;
  r := CALL_WITH_CATCH(function() return IdGroup(G); end, []);
  if not r[1] then
    return fail;
  fi;
  id := r[2];
  return Concatenation("N", String(id[1]), "_", String(id[2]));
end;

IsSemiabelianGroup := function(G)
  local n, key, phi, abNormals, maxReps, A, M;

  if not IsGroup(G) or not IsFinite(G) then
    Error("IsSemiabelianGroup: finite group expected");
  fi;

  n := Size(G);
  if n = 1 then return true; fi;
  if IsAbelian(G) then return true; fi;
  if not IsSolvableGroup(G) then return false; fi;
  if IsNilpotentGroup(G) and NilpotencyClassOfGroup(G) <= 2 then
    return true;
  fi;

  key := SemiabelianCacheKey(G);
  if key <> fail and IsBound(SEMIABELIAN_CACHE.(key)) then
    return SEMIABELIAN_CACHE.(key);
  fi;

  phi := FrattiniSubgroup(G);

  abNormals := ShallowCopy(Filtered(NormalSubgroups(G),
    N -> Size(N) > 1 and Size(N) < n and IsAbelian(N) and not IsSubgroup(phi, N)
  ));

  if Length(abNormals) = 0 then
    if key <> fail then SEMIABELIAN_CACHE.(key) := false; fi;
    return false;
  fi;

  Sort(abNormals, function(a,b) return Size(a) < Size(b); end);
  abNormals := Reversed(abNormals); # descending for fast witness

  maxReps := ShallowCopy(MaximalSubgroupClassReps(G));

  Sort(maxReps, function(a,b) return Size(a) < Size(b); end);
  maxReps := Reversed(maxReps);

  for A in abNormals do
```

```
    for M in maxReps do
      if Size(ClosureGroup(A, M)) = n then
        if IsSemiabelianGroup(M) then
          if key <> fail then SEMIABELIAN_CACHE.(key) := true; fi;
          return true;
        fi;
      fi;
    od;
  od;

  if key <> fail then SEMIABELIAN_CACHE.(key) := false; fi;
  return false;
end;

IsSemiabelian := IsSemiabelianGroup;
```

A.2. Semiabelian non-monomial search.

```
##############################################################################
# semiabelian_nonmonomial_search.g
# Find G with |G|>240 such that IsSemiabelianGroup(G) and not IsMonomialGroup(G)
# Requires semiabelian.g
##############################################################################

if not IsBound(IsSemiabelianGroup) then
  Read("semiabelian.g");
fi;

if not IsBound(MONOMIAL_CACHE) then
  MONOMIAL_CACHE := rec();
fi;
if not IsBound(SEMIABELIAN_NONMONOMIAL_CANDIDATES) then
  SEMIABELIAN_NONMONOMIAL_CANDIDATES := [];
fi;

SafeNumberSmallGroups := function(n)
  local r;
  if n = 1024 then
    Print("# 1024 has 49bn groups - skipping\n");
    return fail;
  fi;
  r := CALL_WITH_CATCH(function() return NumberSmallGroups(n); end, []);
  if not r[1] then
    Print("# NumberSmallGroups(",n,") failed - skipping\n");
    return fail;
  fi;
  return r[2];
end;

SemiCacheKeyMonomial := function(G)
  local r, id;
  if Size(G) > 2000 or Size(G)=1024 then return fail; fi;
  r := CALL_WITH_CATCH(function() return IdGroup(G); end, []);
  if not r[1] then return fail; fi;
  id := r[2];
  return Concatenation("M", String(id[1]), "_", String(id[2]));
end;
```

```
IsNonMonomialVerified := function(G)
  local key, t, tbl, chi;
  if not IsSolvableGroup(G) then return true; fi; # Taketa

  key := SemiCacheKeyMonomial(G);
  if key <> fail and IsBound(MONOMIAL_CACHE.(key)) then
    return not MONOMIAL_CACHE.(key);
  fi;

  if HasIsMonomialGroup(G) then
    if key <> fail then MONOMIAL_CACHE.(key) := IsMonomialGroup(G); fi;
    return not IsMonomialGroup(G);
  fi;

  t := TestMonomialQuick(G);
  if t.isMonomial = false then
    if key <> fail then MONOMIAL_CACHE.(key) := false; fi;
    return true;
  elif t.isMonomial = true then
    if key <> fail then MONOMIAL_CACHE.(key) := true; fi;
    return false;
  fi;

  t := TestMonomial(G);
  if t.isMonomial = false then
    if key <> fail then MONOMIAL_CACHE.(key) := false; fi;
    return true;
  elif t.isMonomial = true then
    if key <> fail then MONOMIAL_CACHE.(key) := true; fi;
    return false;
  fi;

  # scan Irr
  tbl := CharacterTable(G);
  for chi in Irr(tbl) do
    if TestMonomial(chi).isMonomial = false then
      if key <> fail then MONOMIAL_CACHE.(key) := false; fi;
      return true;
    fi;
  od;
  if key <> fail then MONOMIAL_CACHE.(key) := true; fi;
  return false;
end;

# Optimized check: monomial first (cheaper than semiabelian)
IsSemiabelianNonMonomial := function(G)
  # both properties imply solvable - fast filter
  if not IsSolvableGroup(G) then return false; fi;
  # supersolvable => monomial => cannot be non-monomial
  if IsSupersolvableGroup(G) or IsAbelian(G) then return false; fi;
  # expensive lattice last: test non-monomial first
  if not IsNonMonomialVerified(G) then return false; fi;
  # now expensive semiabelian recursion
  if not IsSemiabelianGroup(G) then return false; fi;
  return true;
end;

ValidateCandidate := function(G)
```

```
  Print("=== Candidate ", IdGroup(G), " ", StructureDescription(G), "\n");
  Print(" Size: ", Size(G), " Solvable: ", IsSolvableGroup(G), "\n");
  Print(" IsSupersolvable: ", IsSupersolvableGroup(G), "\n");
  Print(" IsMonomialGroup: ", IsMonomialGroup(G),
        " Has: ", HasIsMonomialGroup(G), "\n");
  Print(" IsSemiabelian: ", IsSemiabelianGroup(G), "\n");
  Print(" Derived length: ", DerivedLength(G),
        " Irr degrees: ", List(Irr(CharacterTable(G)), Degree), "\n");
end;

SearchOrderOptimized := function(n)
  local num, i, G, cands, t0;
  num := SafeNumberSmallGroups(n);
  if num = fail then return []; fi;
  cands := [];
  Print("# Searching order ", n, " (", num, " groups) >240\n");
  t0 := Runtime();
  for i in [1..num] do
    if i mod 500 = 0 then
      Print("# ", n, ":", i, "/", num, " cands:", Length(cands),
            " time ", (Runtime()-t0)/1000.0, "s\n");
    fi;
    G := SmallGroup(n,i);
    if IsSemiabelianNonMonomial(G) then
      Add(cands, [n,i]);
      Add(SEMIABELIAN_NONMONOMIAL_CANDIDATES, [n,i]);
      Print("!!! FOUND order ", n, " id ", i, " : ", IdGroup(G), "\n");
      ValidateCandidate(G);
    fi;
  od;
  Print("# Done order ", n, " found ", Length(cands), "\n");
  return cands;
end;

SearchRangeOptimized := function(a,b)
  local n, all;
  all := [];
  for n in [a..b] do
    Append(all, SearchOrderOptimized(n));
  od;
  return all;
end;
```

A.3. Final check: Verifying Lemma 1.2.

```
FindWitness := function(G)
  local ab, maxs, A, M;
  ab := ShallowCopy(Filtered(NormalSubgroups(G),
          N-> IsAbelian(N) and Size(N)>1 and Size(N)<Size(G)));
  Sort(ab, function(a,b) return Size(a) < Size(b); end);
  ab := Reversed(ab);

  maxs := ShallowCopy(MaximalSubgroupClassReps(G));
  Sort(maxs, function(a,b) return Size(a) < Size(b); end);
  maxs := Reversed(maxs);

  for A in ab do
    for M in maxs do
```

```
      if Size(ClosureGroup(A,M)) = Size(G) and IsSemiabelianGroup(M) then
        return [A,M];
      fi;
    od;
  od;
  return fail;
end;

PrintChain := function(G, depth)
  local w, i, indent;
  indent := "";
  for i in [1..depth] do
    indent := Concatenation(indent, " ");
  od;

  if Size(G)=1 then
    Print(indent, "1 trivial\n");
    return;
  fi;
  if IsAbelian(G) then
    Print(indent, Size(G), " abelian ", StructureDescription(G), "\n");
    return;
  fi;

  w := FindWitness(G);
  if w = fail then
    Print(indent, Size(G), " NO WITNESS FOUND\n");
    return;
  fi;

  Print(indent, Size(G), " = |", Size(w[1]), "|*|", Size(w[2]), "| A=",
        StructureDescription(w[1]), " M=", StructureDescription(w[2]), "\n");
  PrintChain(w[2], depth+1);
end;

# test on your candidate
G := SmallGroup(384,20127);
PrintChain(G,0);
```

A.4. Output of A.2 and A.3.

A.4.1. Output of A.2.

```
# Searching order 384 (56092 groups) >240
!!! FOUND order 384 id 20127 : [ 384, 20127 ]
=== Candidate [ 384, 20127 ]
((C2 x ((C2 x Q8) : C2)) : C2) : C3
Size: 384 Solvable: true
IsSupersolvable: false
IsMonomialGroup: false Has: true
IsSemiabelian: true
Derived length: 3
Irr degrees: [1,1,1,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,8,8,8]
# Done order 384 found 1
```

A.4.2. Output of A.3.

```
gap> G:=SmallGroup(384,20127);;
gap> PrintChain(G,0);
384 = |8|*|96| A=C2 x C2 x C2 M=((C2 x C2 x C2) : (C2 x C2)) : C3
96 = |8|*|24| A=C2 x C2 x C2 M=C2 x A4
24 = |8|*|12| A=C2 x C2 x C2 M=A4
12 = |4|*|3| A=C2 x C2 M=C3
3 abelian C3
```

References

[BEO26] H. U. Besche, B. Eick, and E. O'Brien. SmallGrp, The GAP Small Groups Library, Version 1.7.0. https://gap-packages.github.io/, Aug 2026. GAP package.

[BvdW78] J. C. Bioch and R. W. van der Waall. Monomiality and isoclinism of groups. *Journal für die reine und angewandte Mathematik*, 298:74–88, 1978.

[Den95] Ralf Dentzer. On geometric embedding problems and semiabelian groups. *Manuscripta Mathematica*, 86(2):199–216, 1995.

[Dor67] Larry Dornhoff. M-groups and 2-groups. *Mathematische Zeitschrift*, 100:226–256, 1967.

[GAP26] The GAP Group. *GAP – Groups, Algorithms, and Programming, Version 4.16.1*, 2026.

[Hal40] Philip Hall. The classification of prime-power groups. *Journal für die reine und angewandte Mathematik*, 182:130–141, 1940.

[Kid25] Masanari Kida. On semiabelian groups. *Journal of Group Theory*, 28:697–712, 2025. Communicated by Bettina Eick.

[Leg22] François Legrand. On finite embedding problems with abelian kernels. *Journal of Algebra*, 595:633–659, 2022.

[Mat87] Bernd Heinrich Matzat. Einbettungsprobleme über Hilbertkörpern. In *Konstruktive Galoistheorie*, volume 1284 of *Lecture Notes in Mathematics*, pages 215–268. Springer, Berlin, Heidelberg, 1987.

[Nil26] Nilradical. Nilradical v0: solution and lean proof of kourovka problem 21.68. Mathematical result with Lean formalization, 2026. Nilradical v0.

[Sal82] D. J. Saltman. Generic Galois extensions and problems in field theory. *Advances in Mathematics*, 43(3):250–283, 1982.

[Sch25] Johannes Schmitt. Extremal descendant integrals on moduli spaces of curves: An inequality discovered and proved in collaboration with ai. *arXiv preprint arXiv:2512.14575*, 2025.

[Sto95] Michael Stoll. Construction of semiabelian Galois extensions. *Glasgow Mathematical Journal*, 37(1):99–104, 1995.

[Tho84] J. G. Thompson. Some finite groups which appear as $\mathrm{Gal}\, l/k$, where $k \subseteq \mathbf{Q}(\mu_n)$. *Journal of Algebra*, 89(2):437–499, 1984.

[vdW88] Robert W. van der Waall. Direct products and monomial characters. *Publicationes Mathematicae Debrecen*, 35:149–153, 1988.